3 Task 3: Incorporating SPP with Dynamic I-RRIP

3.1 Goal

Task 3 runs the Stride Pattern Prefetcher (SPP) from Task 1B at the L1D together with the Dynamic I-RRIP L2 policy from Task 2B. The L2 replacement policy is then made *aware* of the lines that SPP brings in, which it can recognise by the packet type `PREFETCH`. The target is a higher IPC than both standalone SPP and Dynamic I-RRIP.

The two mechanisms attack different misses. SPP is a data prefetcher: it removes L1D data misses and does nothing for instructions. Dynamic I-RRIP removes L2 instruction misses and pays for it with extra L2 data misses. The prefetch-aware part decides how the L2 should treat the extra, speculative lines that SPP adds to the L2 traffic.

3.2 Experimental setup

The setup is the same as in Tasks 1B and 2B: ChampSim, 1 core, hashed perceptron branch predictor, 48 KiB 12-way L1D, 512 KiB 8-way L2, 2 MiB 16-way LLC, 25M warm-up followed by 25M measured instructions on the three provided traces. All binaries were built and run with `gcc 7.5`. The only prefetcher is the L1D prefetcher. We evaluate five configurations:

- **Baseline:** no prefetcher, LRU at the L2. This is the reference for speedup and coverage, as in Task 1B.
- **Dynamic I-RRIP:** no prefetcher, the Task 2B policy at the L2 ($T = 0.05$).
- **SPP + LRU:** standalone SPP (`prefetcher/spp.l1d_pref`, unchanged from Task 1B).
- **SPP + Dyn I-RRIP (unaware):** SPP with the Task 2B policy, which treats prefetched lines like any other data line. This is the same binary as the next configuration, run with `DYN_IRRIP_PF_AWARE=0`.
- **SPP + PF-aware Dyn I-RRIP:** this task, `replacement/dyn_irrip.l2c_repl`.

All five configurations were re-run on the current code base (with the Task 2 changes merged), so that they can be compared like for like. The baseline IPCs are identical to those of Tasks 1B and 2B. Our SPP + LRU speedups (1.0087, 1.0065, 1.0160) are within 0.25% of the Task 1B report.

Metrics. IPC is the cumulative IPC of the measured region, and Speedup = $\text{IPC}/\text{IPC}_{\text{baseline}}$. Prefetch accuracy is ChampSim's L1D statistic, *useful load prefetches / prefetches issued to the lower level*, as in Task 1B. Coverage is

$$\text{Coverage} = \frac{\text{L1D load misses}_{\text{baseline}} - \text{L1D load misses}}{\text{L1D load misses}_{\text{baseline}}}.$$

L1D and L2C load MPKI are taken from ChampSim's statistics, and the L2 load MPKI is split into instruction and data loads as in Task 2. Averages use the geometric mean for IPC and speedup and the arithmetic mean for everything else.

3.3 How SPP's prefetches reach the L2

Before designing the policy, we measured what the L2 actually sees from SPP (Table 1).

Trace	L2 policy	SPP prefetch requests	Sent to the L2	L2 prefetch hits (rate)	L2 prefetch fills	Prefetched lines demand-hit in L2	LLC load misses
Trace1	LRU	777 706	64 650	17 493 (27.6%)	45 833	–	293 113
	Dyn (unaware)	778 996	64 781	5 026 (7.9%)	58 397	1 013 (1.7%)	277 907
	Dyn (PF-aware)	779 929	64 706	5 006 (7.9%)	58 364	981 (1.7%)	277 422
Trace2	LRU	522 717	27 901	7 331 (26.4%)	20 391	–	196 869
	Dyn (unaware)	516 212	27 811	2 461 (8.9%)	25 137	490 (1.9%)	199 386
	Dyn (PF-aware)	514 884	27 630	2 437 (8.9%)	24 951	529 (2.1%)	199 287
Trace3	LRU	827 440	34 024	2 890 (8.6%)	30 583	–	63 285
	Dyn (unaware)	826 212	33 975	1 017 (3.0%)	32 426	382 (1.2%)	58 942
	Dyn (PF-aware)	825 041	33 887	1 003 (3.0%)	32 310	388 (1.2%)	58 886

Table 1: SPP's traffic at the L2 over the 25M measured instructions. "Sent to the L2" is ChampSim's *prefetches issued to lower level*. "Prefetched lines demand-hit in L2" counts prefetch fills that later served a demand access in the L2, out of all prefetch fills. It is measured by the policy's statistics, which LRU does not have.

Three observations shaped the design:

1. **The L2 sees little prefetch traffic.** More than 90% of SPP's requests never leave the L1D, because the line is already there or the request is merged. Only 1.1–2.6 prefetches per kilo-instruction reach the L2, compared with 24, 53 and 4.9 L2 demand loads per kilo-instruction on Trace1–3. Any prefetch-specific rule can therefore move IPC only a little.

2. **The L2 copy of a prefetched line is rarely used by a demand access.** Almost all prefetches are `FILL_L1`: 27 274 of 27 901 on Trace2 have the same fill and origin level. The line is placed in the L1D as well, and the L1D serves the demand access. Only 1.2–2.1% of prefetch fills ever get a demand hit in the L2. The rest are evicted unused, or are still resident at the end of the run.

3. **Dynamic I-RRIP lowers the L2 hit rate of prefetch requests** from 26–28% (LRU) to 8–9% on Trace1 and Trace2. Its strict instruction mode evicts data first, so the lines that SPP asks for again are more often gone. Each such miss becomes an LLC access, which makes the prefetch later.

3.4 Design: handling the prefetch packet

The prefetch awareness consists of three rules in `l2c_update_replacement_state`, all keyed on `type == PREFETCH`. Every other packet type is handled exactly as in Task 2B. With no prefetcher, the policy is therefore identical to Task 2B: the no-prefetch runs reproduce the Task 2B results cycle for cycle (Trace1: 106 568 235 cycles in both runs; likewise on Trace2 and Trace3).

Event	Task 2B (unaware)	Task 3 (prefetch-aware)
Prefetch request in a leader set	not counted	not counted (unchanged)
Prefetch fill (miss), any set, any mode	RRPV 2 (as demand data)	**RRPV 1**
Prefetch request hits in the L2	normal promotion	normal promotion (kept)
Demand hit on a prefetched line	normal promotion	normal promotion
Eviction	strict data-first / SRRIP	unchanged; prefetched lines are data

Table 2: Treatment of prefetched lines. The RRPV ordering in instruction mode becomes *code (0) < prefetched data (1) < demand data (2).* In SRRIP mode and in the leader sets, prefetched lines are inserted one step further from eviction than demand fills. In both, lower RRPV means kept longer.

Rule 1: prefetches never enter the set-dueling decision. The leader-set counters count only demand accesses (loads, RFOs, translations). Prefetches are speculative, they are always data, and their number depends on how aggressive SPP currently is, not on what the core is waiting for. Counting them would lower the instruction share r whenever SPP is aggressive, and could switch off instruction priority for the wrong reason. Task 2B already filtered prefetches out of the counters, but without a prefetcher the rule never applied; with SPP it is what keeps the mode decision correct. With SPP running, the leaders still measure $r \approx 0.42$–0.45, 0.66 and 0.17 on Trace1–3, similar to the values without a prefetcher in Task 2B.

Rule 2: a prefetch fill is inserted at RRPV 1. This value is a compromise between two facts from Section 3.3.

- *Do not pin prefetched lines.* Their L2 copy rarely serves a demand access, because the L1D holds its own copy. When we shielded unused prefetched lines from eviction while other data was present, the speedup fell from 1.0301 to 1.0235 (Table 6), and LLC load misses rose (Trace1: 284 727 instead of 277 422). The extra resident prefetches pushed out demand data that was about to be reused.
- *Do not throw them away either.* Inserting them at RRPV 3 ("dead on arrival") lowers the speedup to 1.0284 and raises LLC load misses on every trace. For example, on Trace1 it is 279 727 compared with 277 422. A prefetched line is still a predicted future demand access: SPP's L1D accuracy is 38–53%. The L1D (48 KiB) may drop the line before it is used, and SPP regularly requests the same lines again (the L2 prefetch hits in Table 1). About 2% of the prefetches have a different fill level (`FILL_L2`, issued when the L1D MSHR is more than half full), and for those the L2 copy is the only copy. The replacement interface receives only the packet type, so these cannot be told apart from `FILL_L1` prefetches, and a single mild level of protection is used for all.

RRPV 1 places a prefetched line just above demand-fetched data but below code. A demand-fetched data line has already delivered its first use to the L1D, whereas a prefetched line has not been used yet. It keeps the strict instruction priority of Task 2B intact: code is still never evicted while any data line is present. The sensitivity study in Section 3.6 shows that RRPV 1 is the best of the four possible values, although RRPV 0 is very close.

Rule 3: prefetch hits are promoted like data hits. A prefetch request that hits in the L2 means SPP has just predicted a demand access to that line, so it is treated like a data hit: RRPV 0 in SRRIP mode and one step down in instruction mode. Leaving the RRPV unchanged on a prefetch hit (as some prefetch-aware RRIP proposals do) was measured to be slightly worse (Table 6).

The prefetch rule applies in the leader sets too. The leaders should measure the system as it actually runs, prefetches included. If they ignored prefetches (plain SRRIP insertion), the result was practically identical (1.0297 vs 1.0298 for the matching variants), so we keep one uniform rule.

Hardware cost. The type of the incoming request is already part of the request (MSHR entry), so the prefetch awareness needs no new storage. It needs only a comparator on the type field and a third insertion value. The total cost stays at the Task 2B figure of 50 bits on top of I-RRIP. The `l2_is_pf` bit array in the code feeds only the statistics in Table 1. It is not read by the policy and would not exist in hardware.

3.5 Results

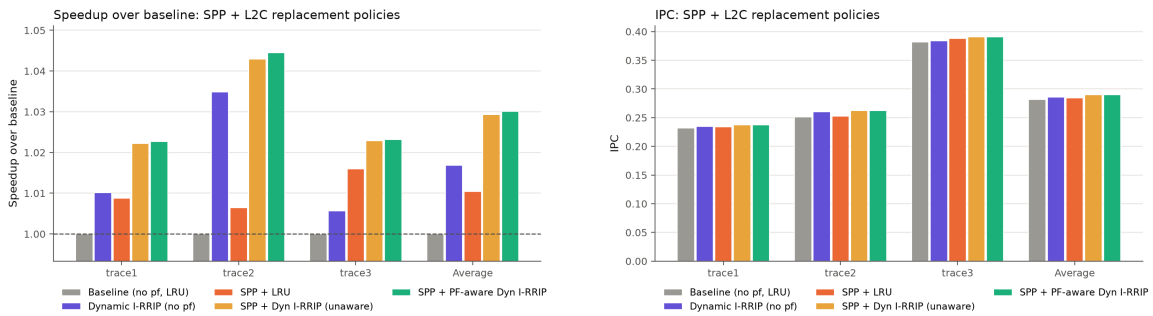
Table 3 and Table 4 report all required metrics, and Figures 1–3 plot them. The raw values are in `results/task3/task3_results.csv`, the tables in `results/task3/task3_tables.md`, and all logs in `results/task3/<configuration>/`.

Trace	Configuration	IPC	Speedup	Accuracy (%)	Coverage (%)	L1D load MPKI
Trace1	Baseline	0.2322	1.0000	–	–	10.679
	Dyn I-RRIP	0.2346	1.0101	–	–	10.680
	SPP + LRU	0.2343	1.0087	53.09	9.63	9.650
	SPP + Dyn (unaware)	0.2374	1.0222	52.93	9.53	9.661
	SPP + PF-aware Dyn	**0.2375**	**1.0226**	52.86	9.53	9.662
Trace2	Baseline	0.2513	1.0000	–	–	13.536
	Dyn I-RRIP	0.2601	1.0349	–	–	13.545
	SPP + LRU	0.2529	1.0065	44.98	2.67	13.175
	SPP + Dyn (unaware)	0.2621	1.0430	44.40	2.58	13.188
	SPP + PF-aware Dyn	**0.2625**	**1.0445**	44.50	2.57	13.189
Trace3	Baseline	0.3819	1.0000	–	–	3.403
	Dyn I-RRIP	0.3840	1.0057	–	–	3.404
	SPP + LRU	0.3880	1.0160	37.94	13.85	2.932
	SPP + Dyn (unaware)	0.3906	1.0229	37.85	13.76	2.935
	SPP + PF-aware Dyn	**0.3907**	**1.0232**	37.97	13.78	2.934
Average	Baseline	0.2814	1.0000	–	–	9.206
	Dyn I-RRIP	0.2861	1.0168	–	–	9.210
	SPP + LRU	0.2843	1.0104	45.34	8.72	8.586
	SPP + Dyn (unaware)	0.2897	1.0293	45.06	8.62	8.595
	SPP + PF-aware Dyn	**0.2899**	**1.0301**	45.11	8.63	8.595

Table 3: IPC, speedup over the baseline, and the L1D prefetch metrics. Accuracy and coverage exist only for the configurations with SPP.

Trace	Configuration	L2C load MPKI	L2 instr load MPKI	L2 data load MPKI	Avg. L2 miss latency (cyc.)	LLC load misses
Trace1	Baseline	17.703	10.368	7.334	140.5	307 044
	Dyn I-RRIP	19.042	9.072	9.970	107.1	292 496
	SPP + LRU	16.825	10.392	6.434	141.9	293 113
	SPP + Dyn (unaware)	17.873	9.043	8.830	108.2	277 907
	SPP + PF-aware Dyn	17.878	9.044	8.834	108.2	**277 422**
Trace2	Baseline	27.560	18.892	8.668	88.9	204 533
	Dyn I-RRIP	21.379	9.264	12.115	82.8	206 936
	SPP + LRU	27.486	19.132	8.354	89.4	196 869
	SPP + Dyn (unaware)	21.041	9.298	11.742	83.4	199 386
	SPP + PF-aware Dyn	**20.844**	**9.096**	11.748	83.7	199 287
Trace3	Baseline	3.960	1.124	2.837	156.9	78 933
	Dyn I-RRIP	3.994	0.727	3.267	109.3	74 944
	SPP + LRU	3.313	1.144	2.169	158.3	63 285
	SPP + Dyn (unaware)	3.308	0.725	2.583	113.1	58 942
	SPP + PF-aware Dyn	**3.307**	**0.723**	2.585	113.1	**58 886**
Average	Baseline	16.408	10.128	6.280	128.8	–
	Dyn I-RRIP	14.805	6.354	8.451	99.7	–
	SPP + LRU	15.875	10.222	5.652	129.8	–
	SPP + Dyn (unaware)	14.074	6.356	7.718	101.6	–
	SPP + PF-aware Dyn	**14.010**	**6.288**	7.722	101.7	–

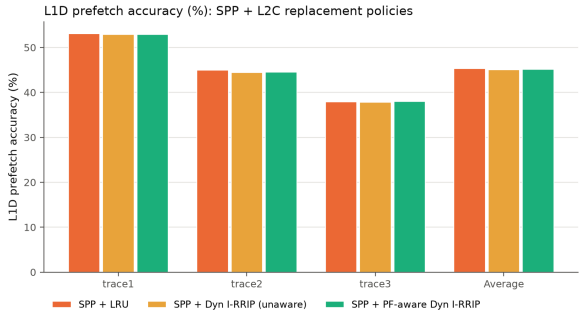
Table 4: L2 demand-load MPKI (total, instruction, data), with the average L2 miss latency and the LLC demand-load misses (loads that went to DRAM) as supporting metrics.



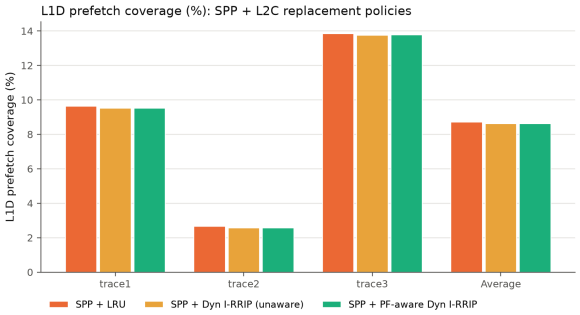
(a) Speedup over the baseline.

(b) IPC.

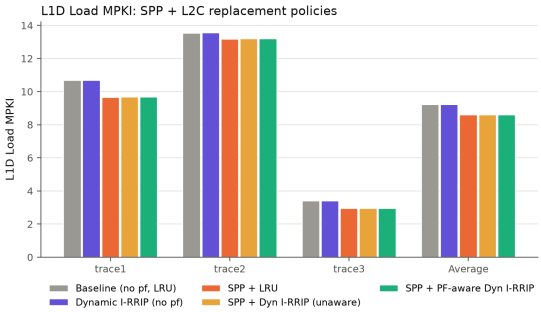
Figure 1: Performance of the five configurations.



(a) L1D prefetch accuracy.

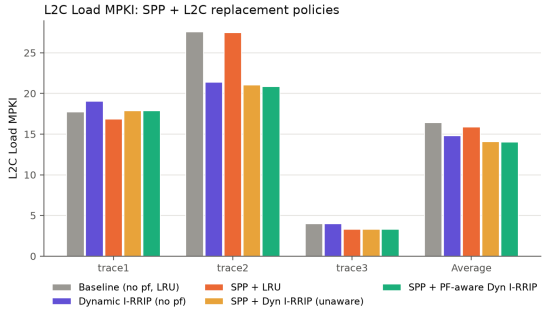


(b) L1D prefetch coverage.

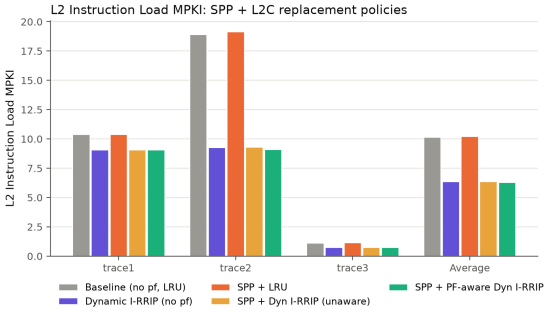


(c) L1D load MPKI.

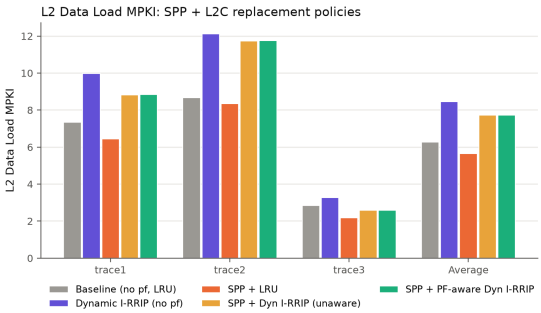
Figure 2: L1D prefetch metrics. The L2 policy hardly changes SPP's accuracy and coverage.



(a) L2C load MPKI.



(b) L2 instruction load MPKI.



(c) L2 data load MPKI.

Figure 3: L2 load MPKI, in total and split by line type.

3.6 Sensitivity study

Both parameters of the final design are read from environment variables, namely `DYN_IRRIP_PF_INS` and `DYN_IRRIP_THRESHOLD`, so one binary covers every point. Table 5 and Figure 4 vary one parameter at a time, keeping the other at its final value.

Value	Trace1	Trace2	Trace3	Geomean speedup	L2C load MPKI	L2 instr MPKI	L2 data MPKI
Prefetch fill insertion RRPV ($T = 0.05$)							
0	1.0226	1.0442	1.0231	1.0299	14.014	6.295	7.718
1 (final)	1.0226	1.0445	1.0232	**1.0301**	14.010	6.288	7.722
2 (= unaware)	1.0222	1.0430	1.0229	1.0293	14.074	6.356	7.718
3	1.0198	1.0430	1.0226	1.0284	14.078	6.361	7.717
Leader-set threshold T (prefetch insertion RRPV 1)							
0.05 (final)	1.0226	1.0445	1.0232	**1.0301**	14.010	6.288	7.722
0.10	1.0226	1.0441	1.0227	1.0297	14.012	6.291	7.721
0.20	1.0220	1.0395	1.0211	1.0275	14.032	6.418	7.613
0.35	1.0217	1.0383	1.0169	1.0256	14.132	6.558	7.574
0.50	1.0090	1.0366	1.0146	1.0200	13.843	7.124	6.719

Table 5: Sensitivity of SPP + prefetch-aware Dynamic I-RRIP. Speedup over the baseline; MPKI columns are means over the three traces. Insertion RRPV 2 with normal promotion is the unaware policy, so that row matches the unaware configuration exactly.

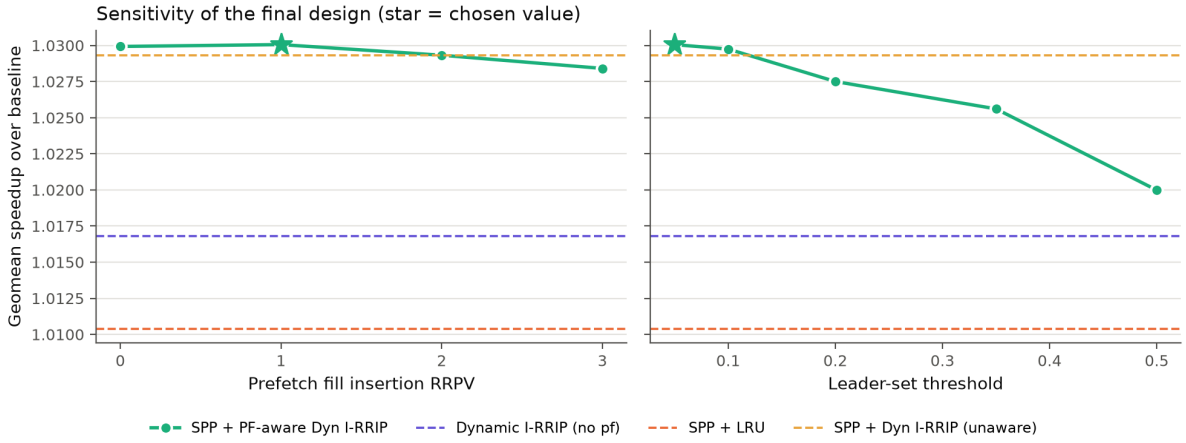


Figure 4: Geomean speedup of the final design against the prefetch insertion RRPV (left) and the leader-set threshold (right). The dashed lines are the other configurations.

Prefetch insertion. The speedup rises monotonically as prefetched lines are given more protection, from RRPV 3 to RRPV 1, and then falls slightly at RRPV 0. RRPV 0 ranks prefetched data equal to code in instruction mode, and the L2 instruction MPKI starts to rise again (6.288 $\rightarrow$ 6.295). The gain of RRPV 1 over the unaware policy (RRPV 2) shows mainly on Trace2: +0.14%, with a lower total L2 load MPKI (21.04 $\rightarrow$ 20.84). On Trace2 this comes from fewer instruction misses (9.30 $\rightarrow$ 9.10 MPKI); the data MPKI is unchanged. We did not isolate why a different treatment of data prefetches lowers instruction misses; presumably it is a second-order effect of the changed eviction order.

Threshold. The Task 2B conclusion still holds with SPP. The speedup falls monotonically as T grows, and $T = 0.05$ is the best value (or tied for best) on every trace. At $T = 0.50$, Trace1 ($r \approx 0.43$) spends 89% of its epochs in SRRIP mode and loses most of its gain (1.0226 $\rightarrow$ 1.0090). Trace3 ($r \approx 0.17$) spends 33–67% of its epochs in SRRIP from $T = 0.20$ upwards. At the chosen $T = 0.05$, all three traces stay in instruction mode for the whole measured region.

3.7 Analysis

The goal is met on every trace. SPP + prefetch-aware Dynamic I-RRIP has the highest IPC on all three traces. The geometric-mean speedup is 1.0301: +1.31% over Dynamic I-RRIP alone (1.0168) and +1.95% over standalone SPP (1.0104). Per trace, it is +1.2%, +0.9% and +1.7% over Dynamic I-RRIP, and +1.4%, +3.8% and +0.7% over SPP.

The two mechanisms are complementary. The product of the two individual speedups is $1.0168 \times 1.0104 = 1.0274$, and the combination reaches 1.0293 even without awareness, so the benefits add up rather than overlap. They remove different misses:

- SPP removes L1D data misses (coverage 2.6–13.8%) and does not change the L2 instruction MPKI.
- Dynamic I-RRIP cuts the L2 instruction MPKI by 38% on average (10.22 $\rightarrow$ 6.29 with SPP). It does this at the cost of L2 data misses (5.65 $\rightarrow$ 7.72 MPKI with SPP).
- SPP softens that cost. With SPP, the L2 data MPKI under Dynamic I-RRIP is 7.72, compared with 8.45 without a prefetcher. That reduction (0.73) is slightly larger than what SPP achieves under LRU (6.28 $\rightarrow$ 5.65).

The L2 miss latency stays at Dynamic I-RRIP's low level, 101.7 cycles against 129.8 for SPP + LRU. LLC load misses (DRAM trips) are the lowest of all configurations on Trace1 and Trace3.

The prefetch awareness adds a small but consistent gain. Compared with SPP plus the unaware Task 2B policy, the prefetch-aware policy is faster on every trace: Trace1 +0.04%, Trace2 +0.14%, Trace3 +0.03%, geomean 1.0293 $\rightarrow$ 1.0301. It also has fewer LLC load misses on every trace, and lower total L2C and instruction MPKI on average. The gain is small for the reason found in Section 3.3: SPP sends only 1.1–2.6 prefetches per kilo-instruction to the L2, and 98% of their L2 copies are never demanded there. No rule for those lines can change much. The sensitivity and exploration results show that the *direction* of the rule matters more than its size. A policy that guesses wrongly in either direction is clearly worse than the unaware one: dead-on-arrival insertion loses 0.09%, and pinning prefetches loses 0.56%.

SPP's accuracy and coverage are almost independent of the L2 policy. Accuracy and coverage change by at most 0.6 and 0.1 percentage points between the three SPP configurations (Table 3). SPP trains on L1D accesses and throttles on its own L1D accuracy, so the L2 policy changes when a prefetch completes (L2 hit or LLC access), not which lines are prefetched. The speedup of the combined design therefore comes from the L2 side, as intended.

3.8 Design exploration

Before fixing the design, we evaluated a range of prefetch-handling options on all three traces with an exploration build of the policy, in which every option sits behind an environment variable

(Table 6).

Prefetch handling (SPP + Dyn I-RRIP, $T = 0.05$)	Trace1	Trace2	Trace3	Geomean
Unaware (Task 2B policy)	1.0222	1.0430	1.0229	1.0293
Fill RRPV 3, prefetch hit not promoted	1.0196	1.0429	1.0225	1.0283
Fill RRPV 3, prefetch hit promoted	1.0198	1.0430	1.0226	1.0284
Fill RRPV 2, prefetch hit not promoted	1.0219	1.0430	1.0229	1.0292
Fill RRPV 2, unused prefetches at RRPV 3 evicted first	1.0218	1.0428	1.0228	1.0291
Adaptive: RRPV 2/3 by L2 usefulness of recent prefetch fills	1.0199	1.0429	1.0226	1.0284
Fill RRPV 1, prefetch hit not promoted	1.0220	1.0443	1.0232	1.0298
with leader sets ignoring prefetches	1.0218	1.0444	1.0232	1.0297
Fill RRPV 1, prefetch hit promoted (final)	1.0226	1.0445	1.0232	**1.0301**
adaptive: RRPV 3 when recent L1D accuracy $< 40\%$	1.0225	1.0443	1.0228	1.0298
unused prefetches never evicted while other data exists	1.0132	1.0402	1.0173	1.0235
Fill RRPV 0, prefetch hit not promoted	1.0227	1.0444	1.0229	1.0299
Fill RRPV 0, prefetch hit promoted	1.0226	1.0442	1.0231	1.0299

Table 6: Prefetch-handling variants, as speedup over the baseline. Raw numbers, the legend and all logs are in `results/task3/design_exploration/`. The final design was also re-run with a rebuilt binary, and the results were identical.

Adaptive insertion did not pay off. A natural idea is to set the insertion depth from the measured usefulness of prefetches. Measured at the L2 (prefetched lines that get a demand hit in the L2), the usefulness is only 1–2% (Table 1) and never reached the 25% threshold in any epoch, so the adaptive policy always picks RRPV 3 and behaves like dead-on-arrival insertion. Measured at the L1D (SPP's own accuracy, 38–53% on average), the per-epoch accuracy hovers around a 40% threshold: it is above it in 86%, 42% and 39% of the epochs on Trace1–3. The policy then alternates between RRPV 1 and RRPV 3 and lands between the two fixed policies (1.0298). On these traces neither signal separates good phases from bad ones, so a fixed insertion depth is enough. Adaptive insertion would matter for a prefetcher whose accuracy swings between phases. SPP already throttles its own degree by accuracy (Task 1B), which reduces the need for it.

Writebacks. For completeness we also tried inserting writeback fills at RRPV 3. Dynamic I-RRIP turns many L1D writebacks into L2 misses (68.9% writeback miss rate on Trace2, against 3.0% for SPP + LRU), most likely because data lines are evicted first in instruction mode. This lowered the speedup to 1.0286, so writebacks keep the Task 2B treatment. It is not part of the prefetch handling.

3.9 Limitations

The prefetch-aware rules add only +0.08% geomean on top of the plain combination, because little prefetch traffic reaches the L2 with this L1D prefetcher. The rules would matter more for an L2 prefetcher or a more aggressive L1D prefetcher, but we could not evaluate those within the assignment's configuration. The insertion RRPV and the threshold were chosen on the same three traces they are evaluated on, so the measured gains may be optimistic for unseen workloads. The small instruction-MPKI improvement on Trace2 from prefetch-aware insertion was observed but not explained mechanistically.

3.10 Reproducing the results

```
./build_champsim.sh no  lru       baseline
./build_champsim.sh no  dyn_irrip dyn_irrip
./build_champsim.sh spp lru       spp
./build_champsim.sh spp dyn_irrip spp_dyn_irrip
# the provided traces are gzip files without an extension
mkdir -p traces
for i in 1 2 3; do ln -sf ../Trace$i-00$i traces/trace$i.champsimtrace.gz; done
task3/run_task3.sh traces 12     # 3 traces x (5 configs + 7 sensitivity)
python3 task3/parse_results.py  # task3_tables.md, task3_results.csv
python3 task3/plot_results.py   # results/task3/plots/
# single run; DYN_IRRIP_PF_AWARE=0 gives the unaware
# Task 2B behaviour, DYN_IRRIP_PF_INS sets the RRPV
DYN_IRRIP_PF_INS=1 bin/<spp_dyn_irrip binary> \
    -warmup_instructions 25000000 \
    -simulation_instructions 25000000 \
    -traces traces/trace1.champsimtrace.gz
```